SECURE ENCRYPTOR AND DECRYPTOR

Project report

Submitted to

The Gandhigram Rural Institute

(Deemed to be University)

In partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF SCIENCE

(COMPUTER SCIENCE)

By

ABHISHEK S

(Register Number: 22226003)

Department of Computer Science and Applications

The Gandhigram Rural Institute

(Deemed to be University)

Gandhigram – 624302

Tamil Nadu

India

April 2025

BONAFIDE CERTIFICATE

This is the certify that the project titled **"SECURE ENCRYPTOR AND DECRYPTOR"** is a bonafide record of work carried out by **ABHISHEK S (Reg.No:22226003)** submitted in partial fulfilment of the requirements for the award of Bachelor of Science in Computer Science in Gandhigram Rural Institute (Deemed to be University), Gandhigram, during the period December 2024 to April 2025.

(Dr. M. MARY SHANTHI RANI)
Internal Guide

(Dr. P. KALAVATHI)
Head of the Department

Submitted for the Viva – Voce examination held on ___ / ___ / ____.

Internal Examiner

External Examiner

DECLARATION

I hereby declare that this project work title **"SECURE ENCRYPTOR AND DECRYPTOR"** is a record of an original work done by me under the supervision and guidance of **Dr. M. MARY SHANTHI RANI, MCA., M.Phil., Ph.D.,** and that this project work has not formed the basis for the award of any Degree/Diploma/Associate-ship/Fellowship or similar title to any candidate of any other University.

PLACE: Gandhigram **(ABHISHEK S)**

DATE :

ACKNOWLEDGEMENTS

First and foremost, I would like to thank the Almighty for blessing me with physical and mental strength for doing this project.

I sincerely thank **Dr. P. Kalavathi, MCA., M.Phil., Ph.D.,** Professor and Head, Department of computer science and applications, The Gandhigram rural institute (Deemed to be university), Gandhigram for the motivation and valuable suggestions given to me and her support has encouraged me in the successful completion of this project in time.

Faculty members of the department, Resarch Scholars and the non – Teaching staff of the department for their help and support to complete this project.

It is also my duty to express my heartfelt thanks to my parents, friends and those who have directly and indirectly helped me in this project work.

(ABHISHEK S)

22226003

TABLE OF CONTENTS

CHAPTER NO.	CONTENTS	PAGE NO.
1	**INTRODUCTION**	**01**
	1.1 Abstract	01
2	**SYSTEM ANALYSIS**	**02-08**
	2.1 Existing System	02
	2.2 Proposed System	03
	2.3 Feasibility study	04
	2.4 User Requirements	05
	2.5 Functional Requirements	06
	2.6 Non-Functional Requirements	07
	2.7 System Constraints	08
3	**SYSTEM REQUIREMENT**	**09-12**
	3.1 Hardware Requirement	09
	3.2 Software Requirement	09
	3.3 Software Description	10
4	**SYSTEM DEVELOPMENT**	**13-15**
	4.1 Modules Description	13
5	**ENCRYPTION AND DECRYPTION ALGORITHMS**	**16-23**
	5.1 Introduction	16
	5.2 Encryption	17
	5.3 Decryption	18
	5.4 Types of Algorithms	19
	5.5 DES Algorithm	20

	5.6 3DES Algorithm	21
	5.7 AES Algorithm	22
6	**SYSTEM TESTING**	**24-26**
	6.1 Functionality Testing	24
	6.2 Encryption/Decryption Testing	24
	6.3 Forms Testing	25
	6.4 HTML/CSS Validation	25
	6.5 Navigation Testing	25
	6.6 Content Testing Checklist	26
	6.7 UI Testing	26
	6.8 Compatibility Testing	26
7	**CONCLUSION AND FUTURE ENHANCEMENT**	**27-28**
	7.1 Conclusion	27
	7.2 Future Enhancement	28
	BIBLOGRAPHY	**29**
	APPENDIX	**30-54**
	Appendix A: Output Screenshot	30
	Appendix B: Code	44

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 ABSTRACT

In today's digital world, safeguarding sensitive information from unauthorized access is crucial. This project tackles that by building a secure **Encryption and Decryption Web Application** to protect digital assets. Unlike basic systems that focus on text-only encryption, this app supports multiple formats—**TXT, DOCX, PPT, XLSX, PDF, image, audio, and video files**—offering a modern and versatile solution. It provides a **user-friendly platform** that simplifies securing different data types while ensuring strong security standards.

The app uses front-end and back-end technologies. The front-end, built with **HTML and CSS**, offers a clean interface for users to select files, enter keys, and start encryption/decryption. **JavaScript** handles user interaction, data processing, and encryption algorithms like **AES, DES, and 3DES** for robust data security. Special attention is given to **secure key management**, vital for protecting encrypted data. With **cross-browser compatibility**, the app works reliably on various platforms.

It efficiently handles different file sizes and formats, providing smooth performance with clear feedback and easy downloads. By integrating strong encryption into a practical web system, it meets today's data protection demands. Unlike limited solutions, this tool proves flexible and strong in real-world use. Ultimately, it ensures a **secure, accessible, and efficient** way to protect sensitive digital data.

SYSTEM ANALYSIS

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Current methods for securing data often involve complex software installations or reliance on third-party services. These solutions may not always be convenient or accessible to all users. Additionally, existing systems might focus on only one type of data, lacking the versatility to handle images, text, and documents within a single platform.

In contrast to the proposed system, the existing methods of encrypting and decrypting data often present limitations in terms of accessibility, ease of use, and security. Some of the limitations of existing systems include:

- **Manual Encryption/Decryption:**

  Users may rely on manual application of encryption algorithms or command-line tools, which can be time-consuming and error-prone. Mistakes in applying the algorithm or managing keys can lead to data breaches or loss.

- **Limited Accessibility:**

  Encryption and decryption tools or services may not be readily accessible or user-friendly for all users. Some solutions might require specialized software or technical expertise, making them inconvenient for users with limited technical skills or those needing quick encryption/decryption on different devices.

- **Key Management Challenges:**

  Existing systems may present challenges in secure key generation, storage, and distribution. Inadequate key management practices can compromise the security of encrypted data, making it vulnerable to unauthorized access or decryption.

- **Format Compatibility Issues:**
  Some encryption tools might have limitations in terms of the types of files or data formats they support. This can lead to compatibility issues and require users to convert files or use multiple tools for different encryption needs.

2.2 PROPOSED SYSTEM:

This project proposes a comprehensive system for encrypting and decrypting images, text, and documents using a combination of front-end and back-end technologies.

Encryption:

Data will be encrypted using robust encryption algorithms (e.g., AES encryption for text and documents, and appropriate methods for image encryption). The system will allow users to select the type of data they wish to encrypt.

Decryption:

Authorized users will be able to decrypt the data using appropriate decryption keys or methods.

User Interface:

A user-friendly web interface will be developed using HTML and CSS to facilitate easy interaction with the encryption and decryption functionalities.

Back-end Logic:

JavaScript will be used to handle the encryption and decryption processes, manage user interactions, and ensure data integrity.

2.3 FEASIBILITY STUDY:

Feasibility is an important phase in the software development process. It enables the developers to have an assessment of the product being developed. It refers to the feasibility study of the product in terms of outcomes of the product. Operational requirements are needed for implementing it. A feasibility study should be performed based on various criteria and parameters. The various feasibility studies are:

- Technical feasibility
- Operational feasibility
- Economic feasibility

2.3.1 Technical Feasibility

- **Algorithm Implementation and Security:** Implementing robust encryption and decryption algorithms (e.g., AES) requires strong cryptographic expertise. Ensuring the correct implementation and security of these algorithms is critical to prevent vulnerabilities. Compatibility issues between different encryption methods or libraries may need to be addressed.

- **Scalability:** The system should be designed to handle future growth in terms of user volume, data volume, and the size of files being encrypted/decrypted. Scalability considerations include software architecture design, efficient algorithm selection, and resource management.

2.3.2 Economic Feasibility

- **Cost-Benefit Analysis:** Conducting a thorough cost-benefit analysis to evaluate the financial implications of implementing the proposed system. Factors to consider include development costs, software/library acquisition (if any), infrastructure costs (if cloud-based), and maintenance expenses.

- **Return on Investment (ROI):** Assessing the expected ROI over a predefined period, considering both tangible and intangible benefits such as improved data security, reduced risk of data breaches, and increased user trust.

2.3.3 Operational Feasibility

- **User Training and Adoption:** Providing clear documentation and user-friendly interfaces to ensure smooth adoption of the encryption and decryption system. User feedback and usability testing will be crucial for identifying and addressing any usability issues or concerns related to the encryption/decryption process.

- **Key Management Procedures:** Implementing clear and secure key generation, storage, and handling procedures. Change management strategies may be needed to ensure users understand and adhere to these procedures, as they are critical for the system's security. Communication and user support are key aspects of effective key management.

2.4 USER REQUIREMENTS

- **User Roles:** Identify different user roles, such as:
    - **General Users:** Individuals needing to encrypt/decrypt personal files or communications.
    - **Administrators:** Users responsible for managing encryption keys or system settings (if applicable in a multi-user context).
    - Each role may have specific needs, access levels, and functionalities within the system.

- **User Interface Preferences:** Gather feedback from potential users to design an intuitive and user-friendly interface. Consider factors such as:

    - Ease of file selection.
    - Clarity of encryption/decryption options.
    - Key input methods (e.g., password entry, key file upload).
    - Visual feedback on the encryption/decryption process.
    - Accessibility features to cater to diverse user preferences and requirements.

2.5 FUNCTIONAL REQUIREMENTS

- **Encryption Functionality:** Implement functionality to encrypt various types of data, including:
    - Text input.
    - Image files.
    - Audio files.
    - Video files.
    - Document files.
    - The system should support different encryption algorithms (e.g., AES, if applicable).
- **Decryption Functionality:** Enable users to decrypt previously encrypted data.
    - The system should support decryption of the same data types and encryption algorithms used for encryption.
    - Users should be able to provide the correct key or password to decrypt the data.
- **Key Management:** Implement functionality for:
    - Key generation (if the system supports it).
    - Key input (e.g., password-based, key file upload).
    - Secure storage of keys (if the system handles key storage, which should be done with extreme caution and security best practices).
- **File Handling:**
    - Allow users to upload files for encryption/decryption.
    - Enable users to download the encrypted/decrypted files.
    - Support various file sizes and formats.

- **Error Handling:**
    - Provide informative error messages to the user in case of incorrect key entry, file corruption, or other issues during encryption/decryption.
- **Process Feedback:**
    - Display clear status messages or progress indicators during the encryption and decryption processes.

2.6 NON-FUNCTIONAL REQUIREMENTS

- **Performance:**
    - Ensure the system can handle encryption and decryption operations efficiently, even with large files.
    - Conduct performance testing to optimize processing times and resource utilization.
- **Security:**
    - Implement robust security measures to protect the confidentiality and integrity of encrypted data and encryption keys.
    - This includes using strong encryption algorithms, secure key management practices, and protecting against vulnerabilities such as injection attacks.
- **Scalability:**
    - Design the system to scale horizontally or vertically to accommodate increases in user traffic, data volume, and file sizes over time.
- **Reliability:**
    - Ensure high availability and fault tolerance to minimize downtime and data loss.
    - Implement data integrity checks and backup/recovery strategies.

- **Usability:**
    - Design the user interface to be intuitive and easy to use, with clear instructions and feedback.
    - Conduct usability testing with representative users to identify areas for improvement and refine the user experience.

2.7 SYSTEM CONSTRAINTS

- **Data Privacy Regulations:**
    - Ensure compliance with data privacy regulations (e.g., GDPR, if applicable) when handling user data and potentially sensitive information.
- **Technological Limitations:**
    - Consider the limitations of existing technologies, libraries, and browser capabilities when designing and implementing the system.
    - Address compatibility issues, performance constraints, and resource limitations to ensure the feasibility and effectiveness of the solution.
- **Cryptographic Restrictions:**
    - Be aware of any legal restrictions or best practices regarding the use of specific encryption algorithms or key lengths.
- **Budgetary Constraints:**
    - Allocate resources for development, testing, security audits, and maintenance within budgetary constraints.
    - Prioritize features and functionalities based on their impact and importance to stakeholders.

SYSTEM REQUIREMENTS

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS (CLIENT-SIDE)

- **Processor:** A modern processor capable of running a web browser efficiently. (e.g., Intel Core i3 or equivalent, AMD Ryzen 3 or equivalent)
- **System Type:** 64-bit Operating System (Generally recommended for modern systems)
- **RAM:** 4GB RAM (Minimum), 8GB or more recommended for handling larger files or improved performance.
- **Hard Disk:** Sufficient storage space to download and store files temporarily during the encryption/decryption process. The required space will vary depending on the size of the files being processed.
- **Monitor:** Standard monitor with a resolution capable of displaying the web page correctly.

3.2 SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10 or later, macOS, Linux, or another modern operating system capable of running a web browser.
- **Front End:** HTML, CSS, JavaScript
- **Back End:** This depends on the implementation. If the encryption/decryption is handled entirely client-side, then no back-end is strictly necessary. If server-side processing is involved, then a back-end language and database may be used (e.g., PHP, Python, Node.js, MySQL, PostgreSQL).
- **Web Server:** If server-side processing is used, a web server will be needed (e.g., Apache, Nginx). For client-side only, a web server is only needed for development and testing purposes (e.g., Visual Studio Code's built-in server, or any simple server).

- **Browser:** Chrome, Edge, Firefox, Safari, or another modern web browser with JavaScript enabled.

3.3 SOFTWARE DESCRIPTION

HTML

Hyper Text Markup Language (HTML) is the set of markup symbols or codes inserted into a file intended for display on the internet. The markup tells web browsers how to display a web page's words and images. For example, content could be structured within a set of paragraphs, a list of bulleted points, or using images and data tables.

The main parts of our element are as follows:

1. The opening tag: This consists of the name of the element (in this case, p) wrapped in opening and closing angle brackets. This states where the element begins or starts to take effect, in this case where the paragraph begins.
2. The closing tag: This is the same as the opening tag, except that it includes a forward slash before the element name. This states where the element ends—in this case where the paragraph ends.
3. The content: This is the content of the element, which in this case, is just text.
4. The element: The opening tag, the closing tag, and the content together comprise the element.

In an encryption/decryption project, HTML would be used to:

- Create the user interface for file upload.
- Provide input fields for keys or passwords.
- Display instructions and feedback to the user.
- Create buttons to trigger encryption/decryption actions.
- Provide download links for the output files.

CSS

CSS stands for Cascading Style Sheets. CSS is a simple design language intended to simplify the process of making web pages presentable.

CSS handles the look and feels part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, layout designs, variations in display for different devices and screen sizes, as well as a variety of other effects.

CSS is easy to learn and understand, but it provides powerful control over the presentation of an HTML document. Most commonly, CSS is combined with the markup languages HTML.

Advantages of CSS:

- CSS saves time - You can write CSS once and then reuse the same sheet in multiple HTML pages.
- Easy maintenance - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.
- Superior styles to HTML - CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.
- Platform Independence - The script offers consistent platform independence and can support the latest browsers as well.

In an encryption/decryption project, CSS would be used to:

- Style the user interface elements (buttons, input fields, etc.).
- Control the layout and appearance of the web page.
- Provide visual cues to the user (e.g., highlighting active elements).
- Ensure a consistent and user-friendly design.

JavaScript

JavaScript is a scripting language used to create and control dynamic website content, i.e., anything that moves, refreshes, or otherwise changes on your screen without requiring you to manually reload a web page.

It was created in 1995 by Brendan Eich while he was an engineer at Netscape. It was originally going to be named Live Script but was renamed.

It is designed to run as a scripting language in a host environment, and it is up to the host environment to provide mechanisms for communicating with the outside world.

Application of JavaScript:

- Web development.
- Web Applications.

In an encryption/decryption project, JavaScript would be used to:

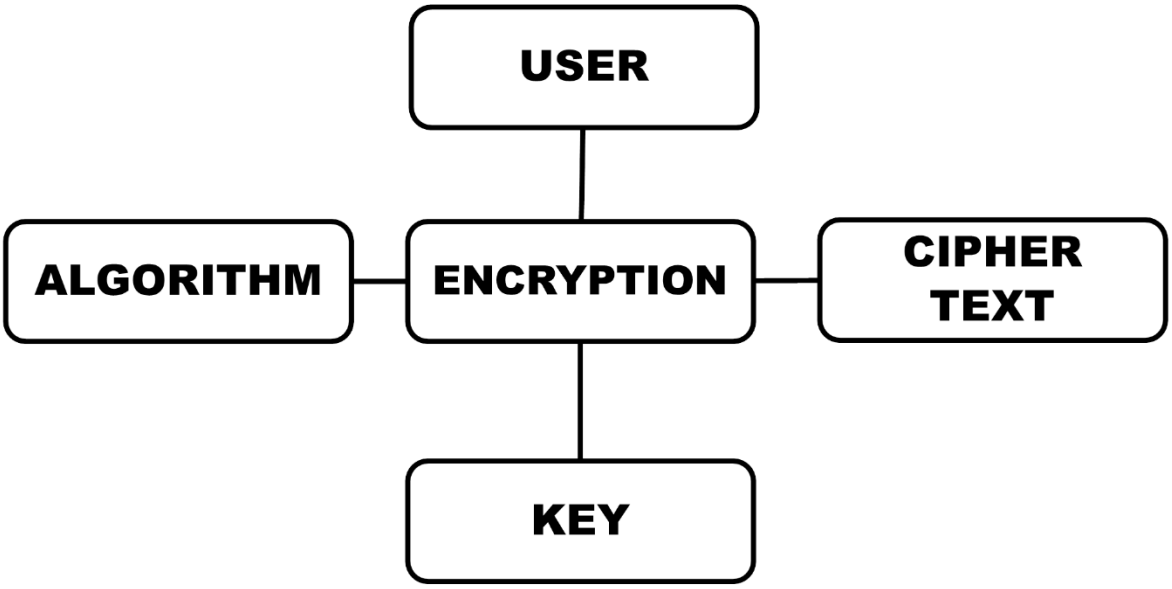
- Handle user interactions (e.g., button clicks, form submissions).
- Implement the encryption and decryption algorithms (if done client-side).
- Manipulate the HTML and CSS to update the user interface.
- Read files uploaded by the user.
- Download the encrypted/decrypted files.
- Perform any necessary data processing or manipulation.

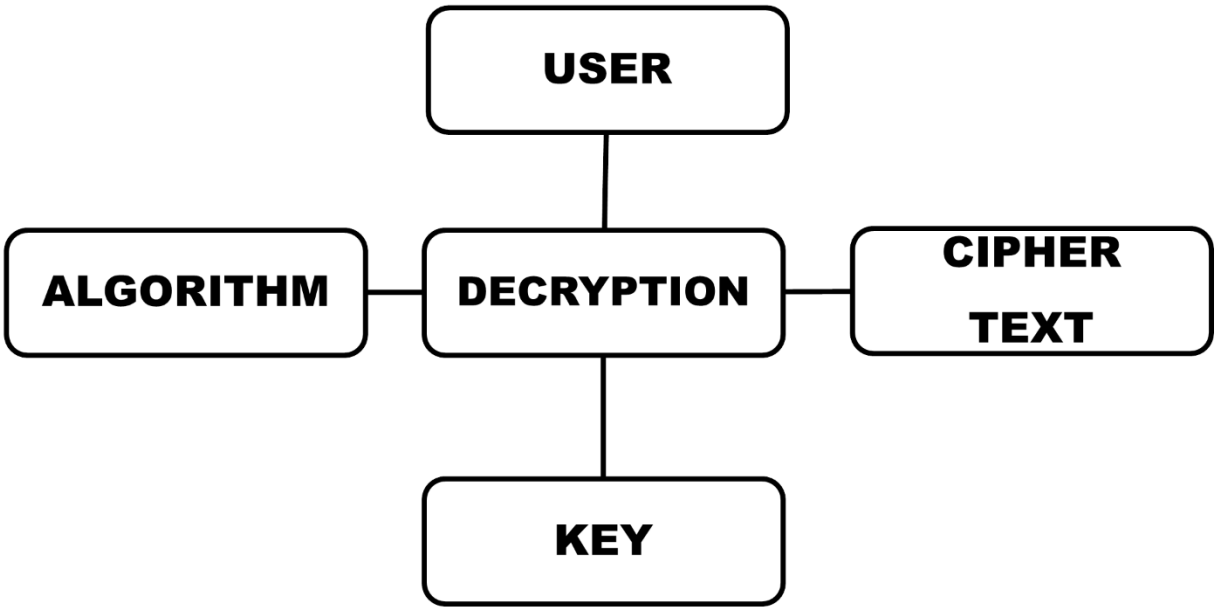
SYSTEM DEVELOPMENT

CHAPTER 4

SYSTEM DEVELOPMENT

4.1 MODULE DESCRIPTION





4.1.1 USER MODULE

- **User Interface:** Design the user interface to be intuitive and user-friendly, with clear instructions and prompts.
- **File Selection:** Implement functionality for users to select the files they want to encrypt or decrypt.
- **Input Handling:** Handle user input for encryption keys or passwords, ensuring secure input methods.

4.1.2 ENCRYPTION MODULE

- **Algorithm Selection:** If the system supports multiple encryption algorithms, provide a way for users to select their preferred algorithm.
- **Encryption Process:** Implement the chosen encryption algorithm to transform the user's data into an encrypted format.
- **Output Generation:** Generate the encrypted output, which could be a downloadable file or displayed text.

4.1.3 DECRYPTION MODULE

- **Decryption Process:** Implement the corresponding decryption algorithm to transform the encrypted data back into its original format.
- **Key Verification:** Verify the decryption key or password provided by the user to ensure it is correct.
- **Output Generation:** Generate the decrypted output, providing the user with access to their original data.

4.1.4 KEY MODULE

- **Key Generation (If Applicable):** If the system supports key generation, provide functionality for users to generate encryption keys.

- **Key Input:** Implement secure methods for users to input their encryption/decryption keys or passwords.
- **Key Storage (If Applicable):** If the system stores keys (which should be handled with extreme caution), implement secure storage mechanisms.

4.1.5 ALGORITHM MODULE

- **Algorithm Implementation:** Implement the encryption and decryption algorithms, ensuring they are correctly implemented and secure.
- **Algorithm Management:** If multiple algorithms are supported, manage their selection and execution.

4.1.6 CIPHERTEXT MODULE

- **Ciphertext Handling**: Manage the encrypted data (ciphertext) securely throughout the system, ensuring it is not altered or exposed unintentionally.
- **Data Integrity Verification**: Implement mechanisms to verify the integrity of the ciphertext, ensuring it has not been tampered with or corrupted during transmission or storage.
- **Ciphertext Storage (If Applicable)**: If the system stores ciphertext files, ensure they are securely stored, with access controls and encryption for added protection.
- **Transmission Security**: If the ciphertext is transmitted over networks, implement secure transmission protocols (e.g., HTTPS, SSL/TLS) to prevent interception or unauthorized access.
- **Format Standardization**: Ensure the ciphertext follows a consistent format or standard, which makes it easier to process during decryption and supports interoperability between different modules or systems.

ENCRYPTION AND DECRYPTION ALGORITHMS

CHAPTER 5

ENCRYPTION AND DECRYPTION ALGORITHMS

5.1 INTRODUCTION

In today's digital world, data security has become an essential requirement for individuals and organizations. With the rapid growth of online transactions, digital communication, and data storage, sensitive information is frequently exposed to potential threats. Unauthorized access, data breaches, and cyber-attacks are common risks that can compromise the confidentiality and integrity of digital data.

Encryption and decryption are fundamental techniques used to secure data and protect it from unauthorized access. Encryption is the process of converting readable data, known as plaintext, into an unreadable format, called ciphertext, using a specific algorithm and an encryption key. This ensures that even if the data is intercepted, it cannot be understood without the correct decryption key.

On the other hand, decryption is the reverse process, where the ciphertext is converted back into its original readable form using the appropriate decryption key. These processes help in maintaining data confidentiality, integrity, and security during storage and transmission.

This chapter explains the concept of encryption and decryption in detail and provides an overview of the different encryption algorithms implemented in this project. It highlights the strengths and limitations of various algorithms, such as AES (Advanced Encryption Standard), DES (Data Encryption Standard), and 3DES (Triple Data Encryption Standard), and explains why these algorithms were chosen for ensuring robust data protection in the proposed system.

5.2 ENCRYPTION

Data encryption is a vital method of protecting sensitive information by converting it from a readable format, known as plaintext, into an unreadable format called ciphertext. This process ensures that only individuals who possess the correct decryption key or password can convert the ciphertext back into its original form. Even if unauthorized users gain access to the encrypted data, they cannot understand or misuse it without the proper key, making encryption a fundamental tool in safeguarding digital information.

There are two primary types of encryptions: asymmetric encryption and symmetric encryption. Asymmetric encryption, also known as public key encryption, uses two separate keys—a public key for encryption and a private key for decryption—allowing for secure communication over untrusted networks. Symmetric encryption, on the other hand, relies on a single key for both encryption and decryption, making it faster but dependent on secure key distribution between the sender and recipient. Additionally, hashing is often used alongside encryption to generate fixed-length hash values that verify data integrity, ensuring the data has not been altered.

Encryption plays a crucial role in maintaining data security by providing authentication, data integrity, and non-repudiation. Authentication verifies the source of the data, ensuring it originates from a trusted party. Data integrity guarantees that the information remains unchanged during transmission, and non-repudiation prevents the sender from denying their involvement in sending the data. While encryption does not prevent unauthorized access to systems, it renders stolen data useless to cybercriminals and other malicious actors, protecting sensitive information from exploitation.

5.3 DECRYPTION

Data decryption is the process of converting encrypted data, known as ciphertext, back into its original readable form, called plaintext. This process requires a decryption key or password and is essential for authorized users to access and understand the protected information. Decryption serves as the counterpart to encryption, ensuring that only individuals with the proper credentials can unlock and use sensitive data that has been secured.

Decryption works by applying a specific key to reverse the encryption algorithm's transformation of plaintext into ciphertext. The decryption key is often unique to the encryption method used, and its security is critical—if unauthorized users gain access to the key, the confidentiality of the data is compromised. There are two primary types of decryptions: symmetric decryption, where the same key is used for both encryption and decryption (as in AES), and asymmetric decryption, where a public key encrypts the data and a private key decrypts it (as in RSA). Symmetric decryption is typically faster but requires secure key distribution, while asymmetric decryption offers better key management flexibility at the cost of speed.

The importance of data decryption lies in its role in maintaining access control and data confidentiality. It ensures that only authorized individuals can read and make use of sensitive data, protecting it from unauthorized interpretation or misuse. Even if encrypted data is intercepted or stolen, without the correct decryption key, the data remains secure and unreadable, safeguarding the privacy and integrity of critical information.

5.4 TYPES OF ALGORITHMS

5.4.1 SYMMETRIC ALGORITHM

Symmetric algorithms use a single key for both encryption and decryption processes. This means the same key that is used to convert plaintext into ciphertext is also used to convert the ciphertext back into plaintext. The primary advantage of symmetric algorithm lies in its simplicity and speed, as the algorithms are less complex and require fewer computational resources compared to asymmetric encryption.

Some of the most commonly used symmetric algorithms include AES (Advanced Encryption Standard), DES (Data Encryption Standard), and 3DES (Triple DES). AES is considered the most secure and efficient among these, offering key lengths of 128, 192, or 256 bits. DES and 3DES are older algorithms, with DES now largely considered obsolete due to its shorter key length and vulnerability to brute-force attacks. 3DES improves upon DES by applying the encryption process three times, increasing security at the cost of speed.

Symmetric encryption is ideal for scenarios where large amounts of data need to be encrypted quickly. It is commonly used in securing file storage, database encryption, and virtual private networks (VPNs). However, one of its main challenges is key management—the encryption key must be securely shared between the sender and the receiver. If the key is intercepted by an unauthorized party, the entire system's security can be compromised.

5.4.2 ASYMMETRIC ALGORITHM

Asymmetric encryption algorithms use a pair of keys—a public key and a private key—to perform encryption and decryption. The public key is freely distributed and used to encrypt data, while the private key is kept secret and used to decrypt the data. This key pair approach eliminates the need for both parties to share a single secret key, addressing one of the major limitations of symmetric encryption.

Popular asymmetric encryption algorithms include RSA (Rivest-Shamir-Adleman), ECC (Elliptic Curve Cryptography), and Diffie-Hellman. RSA is one of the oldest and most widely used algorithms for secure data transmission. ECC is gaining popularity due to its ability to provide strong encryption with smaller key sizes, making it more efficient. Diffie-Hellman is primarily used for securely exchanging cryptographic keys over a public channel.

Asymmetric encryption is commonly used for secure key exchange, digital signatures, and establishing secure communication channels, such as in SSL/TLS protocols used by websites. While it offers enhanced security—especially in situations where two parties have not previously shared secret information—it is slower and more computationally intensive compared to symmetric encryption. Therefore, asymmetric algorithms are often used alongside symmetric encryption to securely exchange keys and establish secure sessions, after which symmetric encryption handles the bulk data encryption.

5.5 DATA ENCRYPTION STANDARD (DES) ALGORITHM

The Data Encryption Standard (DES) is a symmetric-key block cipher developed by IBM in the early 1970s, based on the Lucifer cipher designed by Horst Feistel. It was adopted as a Federal Information Processing Standard (FIPS) by the National Bureau of Standards (NBS) in 1977. DES became one of the most widely used encryption algorithms for securing sensitive but unclassified information. The algorithm uses a 56-bit effective key (originally a 64-bit key, with 8 bits used for parity) and operates on 64-bit blocks of data. Its structure is a balanced Feistel network with 16 rounds of processing, using substitution (S-boxes) and permutation (P-boxes) operations. The encryption process divides data into two halves and applies complex rounds of expansion, key mixing, substitution, and permutation. Decryption is essentially the same process, applied in reverse order using the same algorithm.

DES faced significant scrutiny due to its relatively short key length, which made it vulnerable to brute-force attacks as computing power increased. Despite early collaboration with the NSA, which adjusted the design to resist differential cryptanalysis, the reduction of key size raised concerns. However, there was no evidence of intentional

backdoors. Over time, DES was successfully broken multiple times: in 1997 by the DESCHALL Project using distributed computing, in 1998 by the Electronic Frontier Foundation (EFF) with their custom-built "Deep Crack" machine (which broke DES in 56 hours), and later by cost-effective hardware like COPACOBANA and RIVYERA. Cryptanalysis methods like differential cryptanalysis, linear cryptanalysis, and Davies' attack were all developed and tested on DES, although brute-force attacks remained the most practical approach to breaking it.

Due to its weaknesses, DES was officially replaced by the Advanced Encryption Standard (AES) in 2001 and formally withdrawn in 2005. Triple DES (3DES), which applies the DES algorithm three times to each data block, was introduced as a temporary solution to improve security but at the cost of speed and efficiency. Today, 3DES is also being phased out. Despite its vulnerabilities, DES made a lasting impact on the field of cryptography. It sparked significant academic research and led to the development of modern cryptanalytic techniques. As noted by cryptographer Bruce Schneier, DES "galvanized the field of cryptanalysis" and set the foundation for evaluating the security of future block ciphers.

5.6 TRIPLE DES (3DES) ALGORITHM

Triple Data Encryption Algorithm (TDEA), commonly known as Triple DES (3DES), is a symmetric-key block cipher designed to improve the security limitations of the original Data Encryption Standard (DES). Introduced in 1981, 3DES applies the DES algorithm three times to each data block, enhancing its strength against brute-force attacks. It operates on 64-bit data blocks and typically uses either two or three independent keys, providing effective key lengths of 112 or 168 bits. The encryption process involves three sequential operations: encryption with the first key (K1), decryption with the second key (K2), and encryption again with the third key (K3). This structure provides backward compatibility with DES, allowing legacy systems to transition to a more secure encryption method.

3DES supports three keying options that define its level of security. Option 1 (3TDEA) uses three independent keys, offering the highest security with an effective key length of 168 bits. Option 2 (2TDEA) uses two keys where the first and third keys are the same (K1 = K3), providing 112 bits of effective security—this option was disallowed by NIST in

2015 due to its reduced strength. Option 3, where all three keys are identical (K1 = K2 = K3), effectively reduces 3DES to the same level of security as single DES and is not recommended. Despite improvements over DES, 3DES remains vulnerable to certain cryptographic attacks. Notably, meet-in-the-middle attacks diminish its practical security, and the Sweet32 vulnerability (CVE-2016-2183) highlighted weaknesses related to its small block size, making it susceptible to block collision attacks in some network protocols.

Due to its inherent limitations and the rise of more advanced encryption algorithms, 3DES has been largely phased out in modern cryptographic applications. NIST deprecated 3DES in 2019, with plans to disallow its use entirely by 2024. Many systems that historically relied on 3DES, such as EMV payment cards and older versions of Microsoft Office and Mozilla products, have since transitioned to more secure standards like the Advanced Encryption Standard (AES). OpenSSL, for example, removed 3DES from its default configuration in 2016. While 3DES served as a vital bridge between DES and modern encryption standards, it has now reached the end of its lifecycle. Today, stronger algorithms like AES are recommended for securing sensitive information due to their superior security and efficiency.

5.7 ADVANCED ENCRYPTION STANDARD (AES) ALGORITHM

The Advanced Encryption Standard (AES), also known as Rijndael, is a symmetric-key block cipher developed by Belgian cryptographers Joan Daemen and Vincent Rijmen. It was selected by the U.S. National Institute of Standards and Technology (NIST) in 2001 as a replacement for the aging Data Encryption Standard (DES). AES became effective as the U.S. Federal Information Processing Standard (FIPS) 197. It operates on fixed 128-bit blocks and supports key sizes of 128, 192, or 256 bits, offering flexibility in security levels. Unlike DES, which uses a Feistel network, AES employs a Substitution-Permutation Network (SPN) structure, making it more resistant to cryptanalysis and significantly more efficient in modern computing environments.

AES encryption transforms the input plaintext into ciphertext through a series of well-defined steps. The plaintext is first arranged into a 4×4 byte matrix called the "state." Encryption involves multiple rounds—10, 12, or 14 rounds depending on the key size.

Each round applies four core operations: SubBytes, which substitutes bytes using an S-box; ShiftRows, which shifts rows of the matrix for diffusion; MixColumns, which mixes columns to further spread the data; and AddRoundKey, where the state is XORed with a portion of the encryption key. The final round omits the MixColumns step. AES is designed for high efficiency in both hardware and software implementations, and it performs exceptionally well on modern CPUs, particularly those with AES-NI instructions.

AES has withstood extensive cryptanalysis and remains the global standard for securing sensitive data. It is approved by the NSA for securing classified information, including Top Secret data when using 192 or 256-bit keys. AES-256 offers strong resistance against both classical and quantum attacks, while AES-128 and AES-192 are considered less quantum-resistant due to their shorter key lengths. While theoretical attacks such as biclique and related-key attacks exist, they require unrealistic conditions and remain impractical. AES is widely used in government, financial services, and the technology sector, with standards and validations like NIST's CMVP and CAVP ensuring compliance for secure systems. FIPS 140-2 validation is mandatory for U.S. government systems handling sensitive data, reinforcing AES as the cornerstone of modern encryption.

SYSTEM TESTING

CHAPTER 6

SYSTEM TESTING

6.1 FUNCTIONALITY TESTING:

Functional testing aims to ensure that each function of the website operates in conformance with the requirement specification. Website testing of the functionality shows "What the system does?" Let's try to create the checklist for website functionality testing.

6.2 ENCRYPTION/DECRYPTION TESTING:

- **File Encryption:**
  - Verify that files of various types (text, images, documents) can be successfully encrypted.
  - Test encryption with different file sizes.
  - Check the integrity of the encrypted files.
- **File Decryption:**
  - Verify that encrypted files can be successfully decrypted.
  - Ensure that decryption with the correct key/password restores the original file.
  - Test decryption with incorrect keys/passwords and verify that it fails.
- **Key Input/Management:**
  - Test different methods of key input (e.g., password, key file).
  - If the system generates keys, test the key generation process.
  - If the system stores keys (not recommended for simple encryption), test key retrieval and security.

- **Algorithm Testing:**
    - If multiple encryption algorithms are implemented, test each one individually.
    - Ensure that the selected algorithm is correctly applied.

6.3 FORMS TESTING:

We use forms for interactive communication with our customers. So,

- The input data validity.
- Allowed values for the data field
- Invalid input values for the data field
- Options for forms in which deletion or any other modification of data is possible.

6.4 HTML/CSS VALIDATION:

- HTML syntax errors
- Verify the site is available for search machines
- Verify our web page has an accurate site map in both XML and HTML format.
- Usability testing is aimed to evaluate our web page by testing it with representative users.
- It helps to define user ability to learn to operate, prepare inputs for and interpret outputs of our site.

6.5 NAVIGATION TESTING:

Navigation testing contains the next verifications:

- All pages of our site are understandable and easy to use Buttons, shapes and fields are convenient for use.
- There is access to the Main menu from all pages.

6.6 CONTENT TESTING CHECKLIST:

- There are no grammar and spelling mistakes Images are placed properly with proper sizes.
- Verify optimization of the color palette of the site and font sizes.
- Content should be informative, understandable, structured, and linked.

6.7 UI (USER INTERFACE) TESTING:

- User Interface (UI) testing is provided to verify the graphic user interface of our website meets the specifications.
- Here are some verifications for UI testing of a website: Compliance with the standards of Graphical Interfaces.
- Design elements evaluation: colors, fonts, font sizes, labels, text boxes, text formatting, captions, buttons, lists, icons, links.
- Testing with different screen resolutions.
- Testing the graphical user interface on target devices: smartphones and tablets.

6.8 COMPATIBILITY (CONFIGURATION) TESTING:

- Compatibility (Configuration) testing is performed to test our website with each one of the supported software and hardware configurations:
- OS configuration Browser configuration.
- Cross-platform testing allows evaluating the work of our site in different OS (both desktop and mobile): Windows, iOS/Mac OS, Linux, Android, and BlackBerry etc.

CONCLUSION AND FUTURE ENHANCEMENT

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 CONCLUSION:

The **Encryption and Decryption System** has been successfully developed into a robust, efficient, and error-free solution designed to secure sensitive data. Throughout the development process, special attention was paid to system design, ensuring **data integrity**, eliminating **redundancy**, and maintaining a **user-friendly interface**. The system offers an intuitive experience that allows users to perform encryption and decryption tasks quickly and flexibly. Whether for individuals or organizations, the system enables easy navigation and reliable data protection without requiring advanced technical knowledge.

Extensive **system analysis**, **functional**, and **non-functional testing** were conducted to validate the platform's performance, reliability, and security. The system accurately encrypts and decrypts data while maintaining high levels of confidentiality and integrity under varying conditions. It performs reliably even under heavy loads, demonstrating strong **scalability** and **stability**. By adhering to **security best practices**, the system ensures that users' digital assets are safeguarded against unauthorized access, making it a valuable tool for protecting sensitive information.

In conclusion, the **Encryption and Decryption System** is ready for deployment and practical use by a wide range of stakeholders, including individuals, businesses, and organizations. While the current version meets immediate user needs, future enhancements—such as support for more advanced encryption algorithms, enhanced key management, and integration with cloud storage—can expand its functionality. The system provides a solid foundation for addressing present-day data security challenges and offers a scalable solution for evolving digital security requirements.

7.2 FUTURE ENHANCEMENT:

- **Support for Additional Encryption Algorithms:** Introduce support for a wider range of encryption algorithms, giving users more options for securing their data.
- **Enhanced Key Management:** Implement more advanced key management features, such as key versioning, key rotation and secure key storage options (e.g., integration with hardware security modules or key management systems).
- **Integration with Cloud Storage:** Enable users to directly encrypt and decrypt files to and from cloud storage services (e.g., Google Drive, Dropbox).
- **Automated Encryption/Decryption:** Implement options for automated encryption/decryption of files or data based on predefined rules or schedules.
- **User Access Controls:** If the system is multi-user, implement robust user access controls to manage who can encrypt/decrypt specific data.
- **Audit Logging:** Implement detailed audit logging to track encryption/decryption activities, providing a record of who accessed and modified data.
- **Improved Error Handling and User Feedback:** Enhance error messages and provide more detailed feedback to users during the encryption/decryption process to help them troubleshoot issues.
- **Cross-Platform Compatibility:** Ensure the system is compatible with a wider range of operating systems and devices.
- **Performance Optimization:** Optimize the encryption/decryption algorithms and processes to improve performance, especially for large files.
- **Accessibility Features:** Implement accessibility features to make the system usable for individuals with disabilities.

BIBILOGRAPHY

BIBILOGRAPHY

1. https://www.digitalguardian.com - Offers insights, tools, and resources on data loss prevention (DLP) and enterprise data security.
2. https://www.tutorialspoint.com - A comprehensive platform for learning programming, software tools, and tech concepts with tutorials and examples.
3. https://www.w3schools.com - Popular web platform to learn HTML, CSS, JavaScript, and other web technologies through interactive examples.
4. https://www.wikipedia.org - A free, crowd-sourced encyclopedia covering information on nearly every known topic.
5. https://www.nist.gov - Official U.S. site offering cybersecurity standards, frameworks, and technology guidelines.
6. https://developer.mozilla.org (MDN) - Best for web technologies (HTML, CSS, JavaScript)
7. https://css-tricks.com - Tips, guides, and solutions specifically for CSS and frontend development.

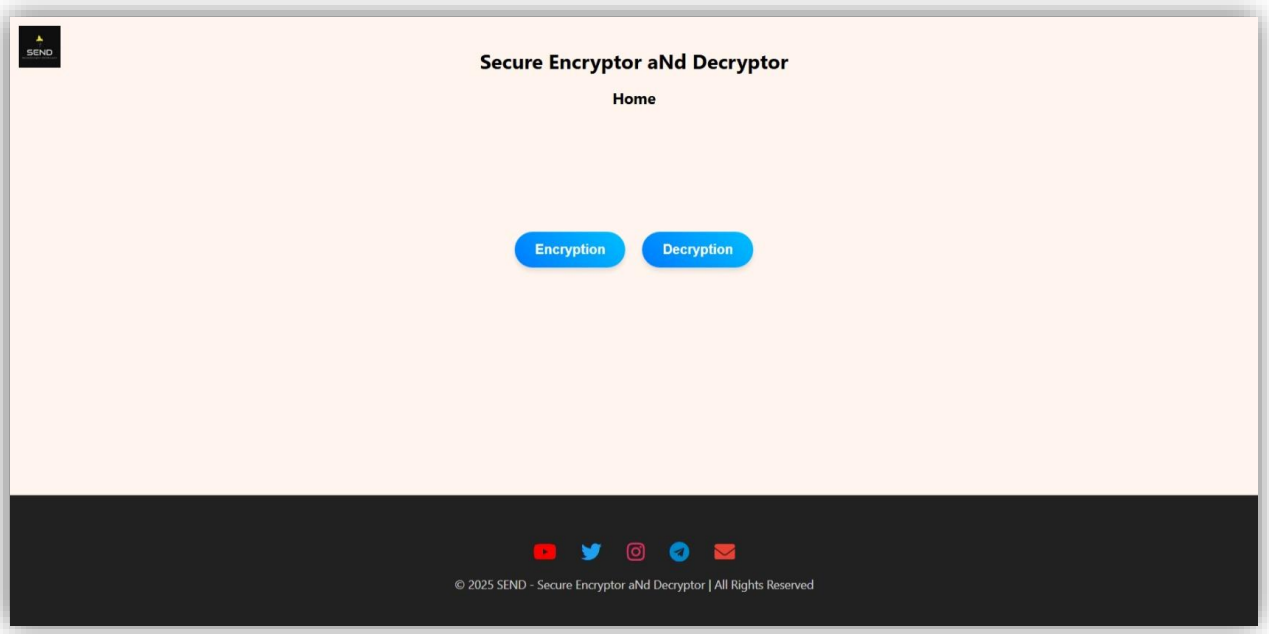
APPENDIX

APPENDIX

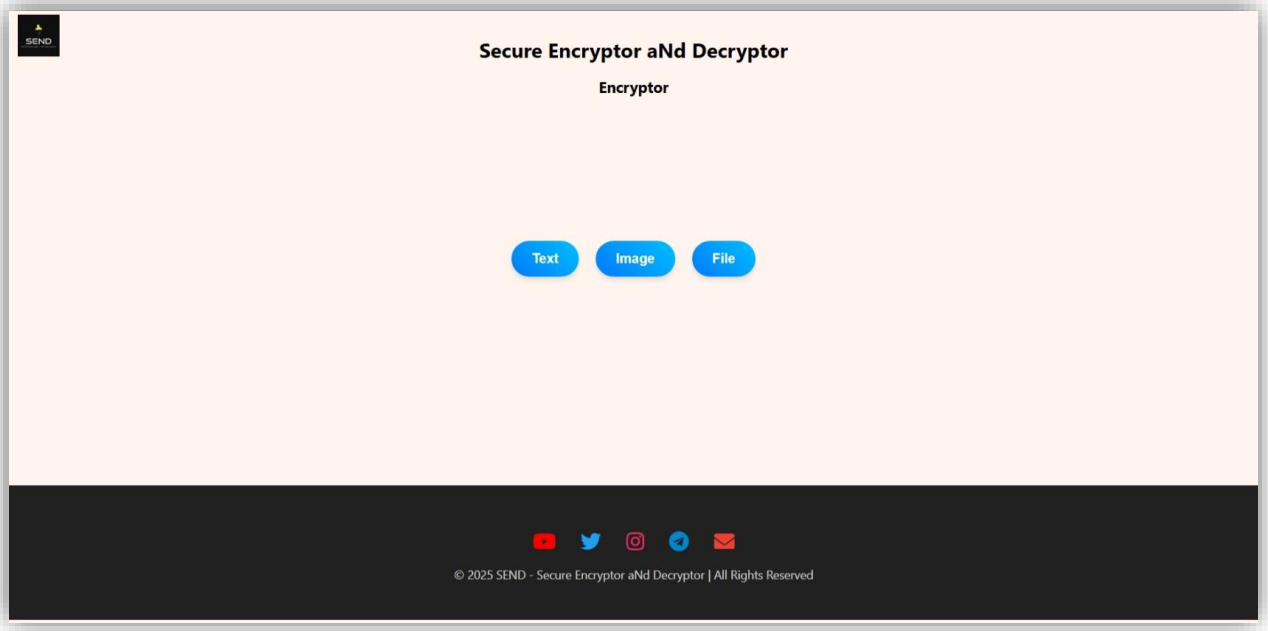
APPENDIX A: OUTPUT SCREENSHOTS

LANDING PAGE:

HOME PAGE:



ENCRYPTION PAGE:



TEXT ENCRYPTION PAGE:

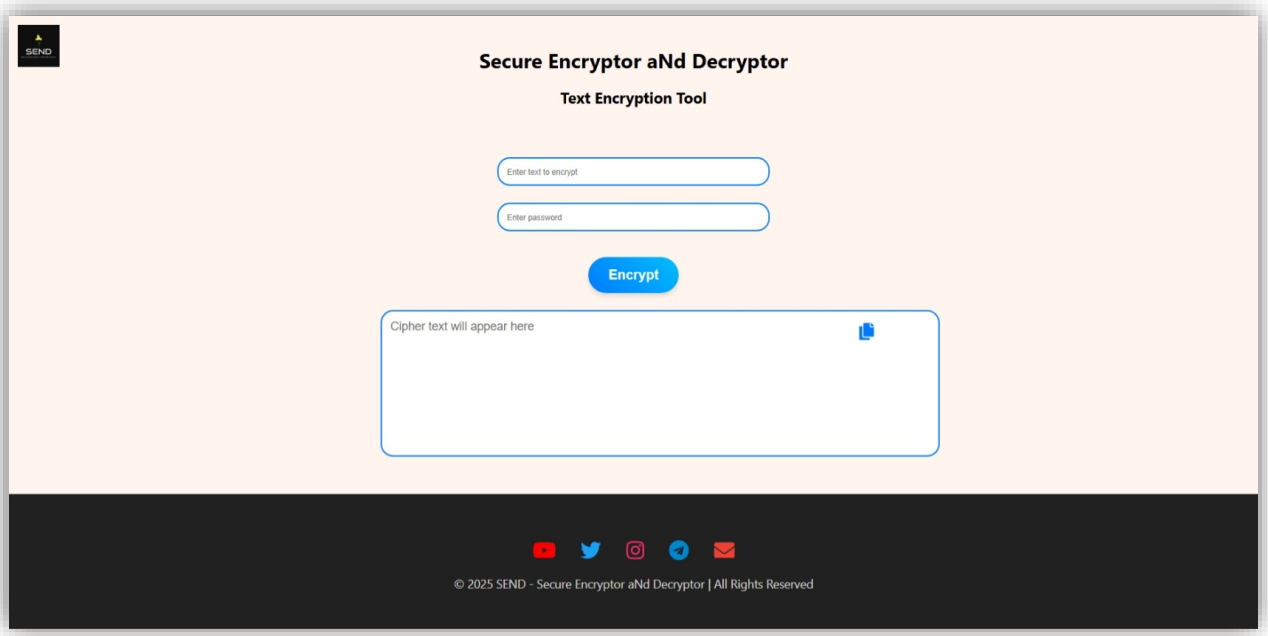
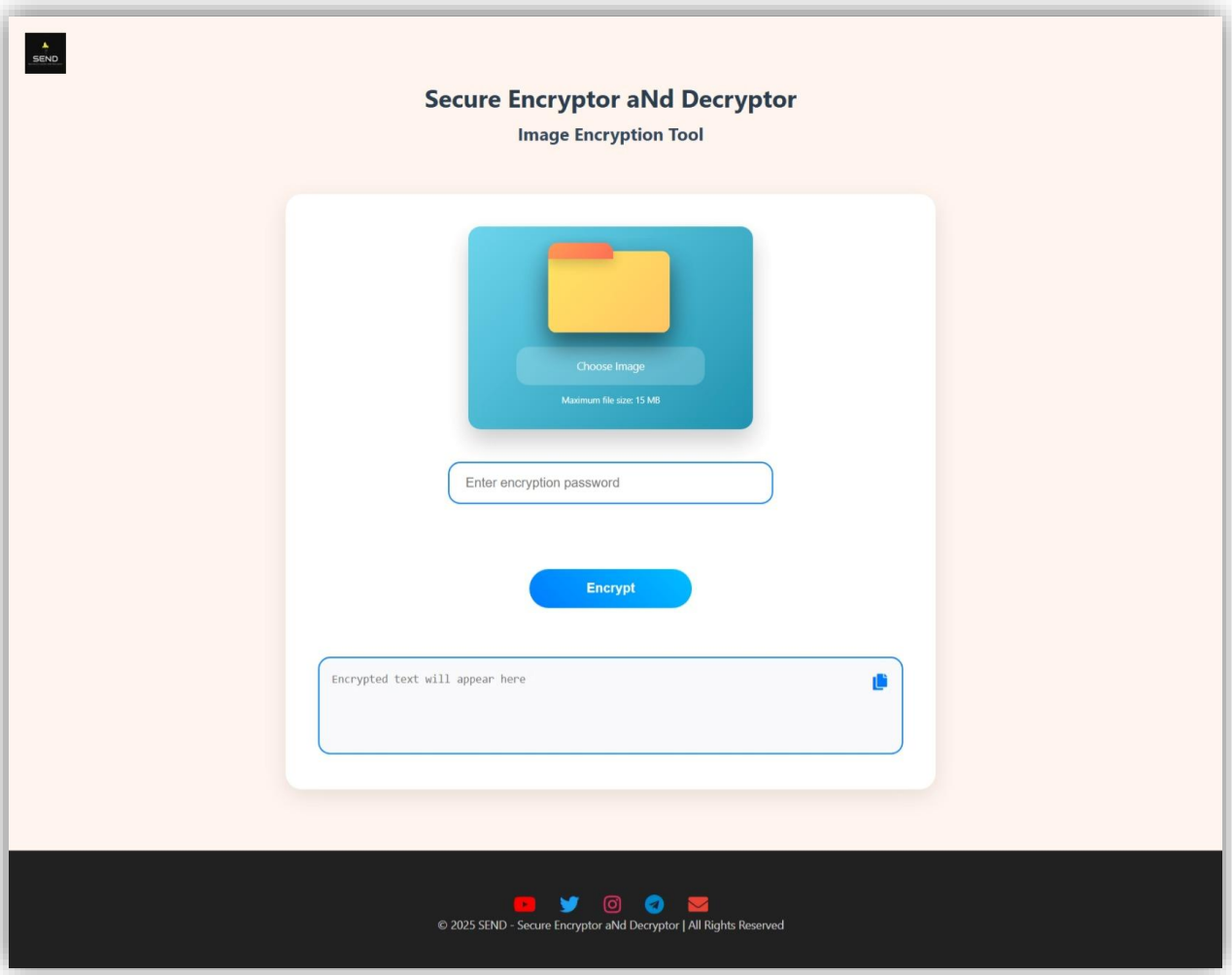
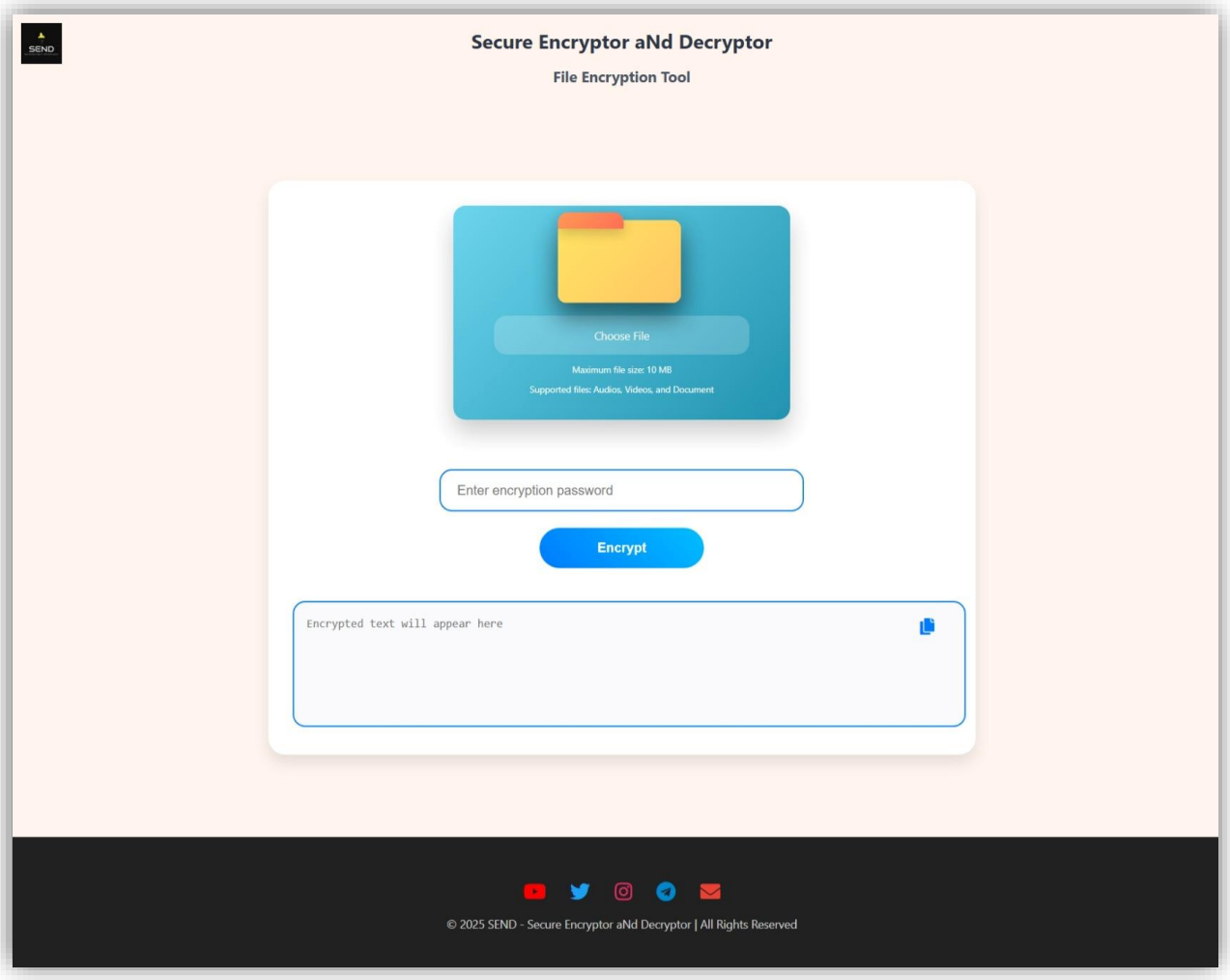


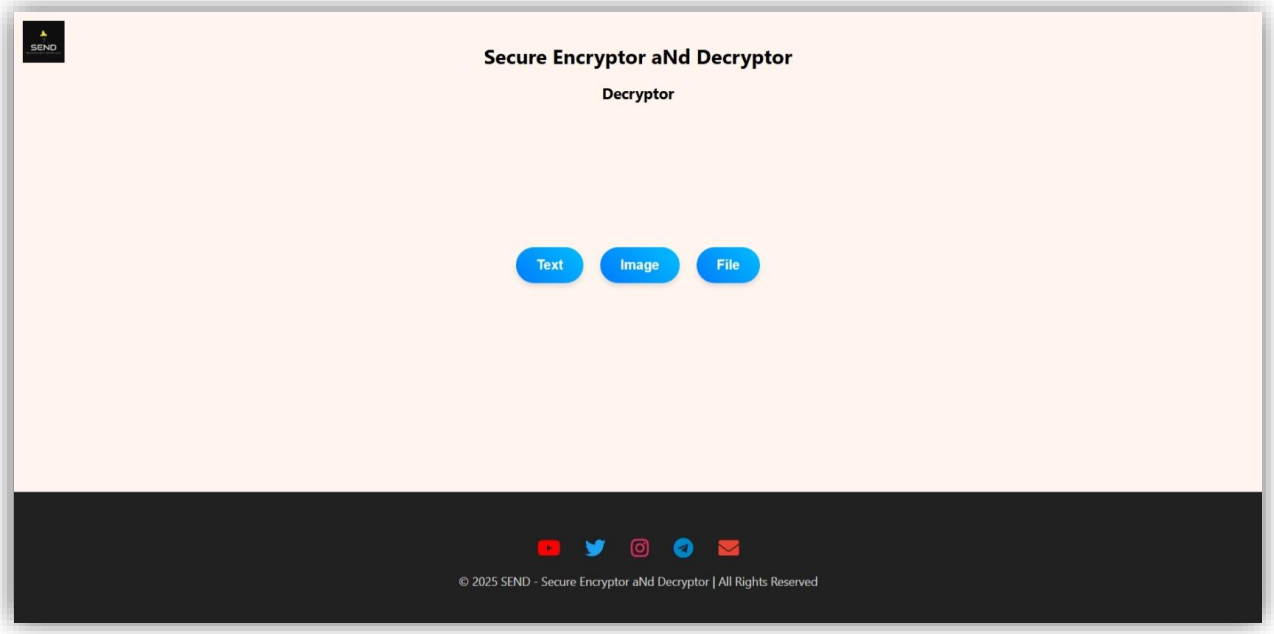
IMAGE ENCRYPTION PAGE:



FILE ENCRYPTION PAGE:



DECRYPTION PAGE:



TEXT DECRYPTION PAGE:

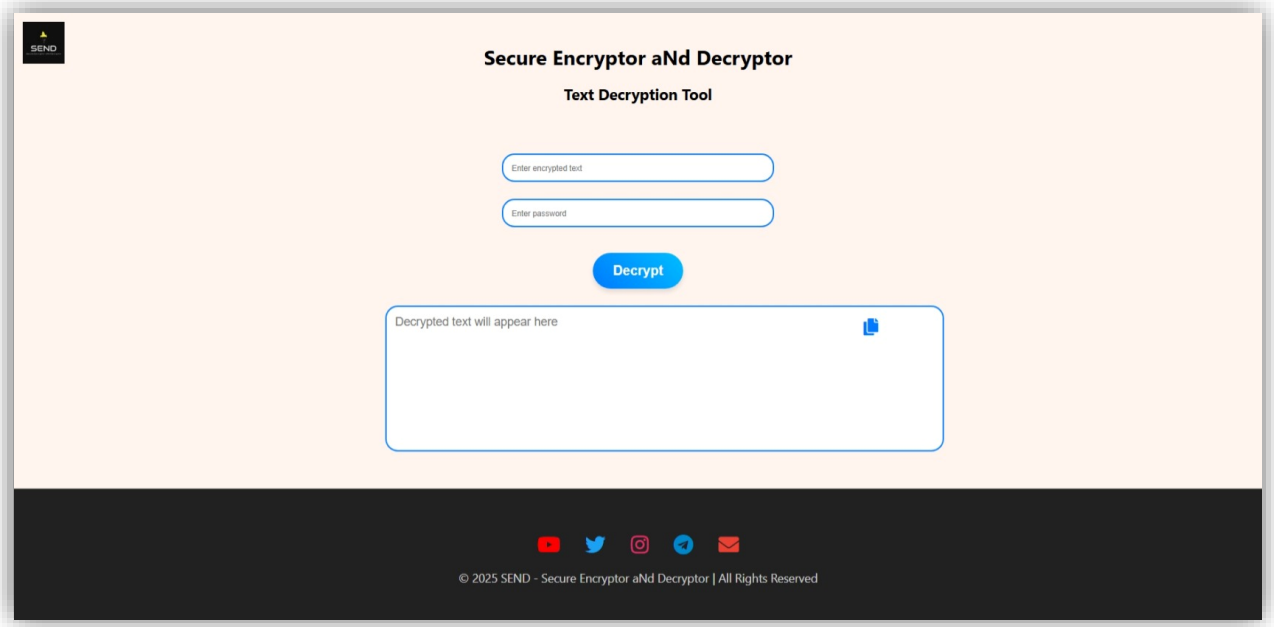
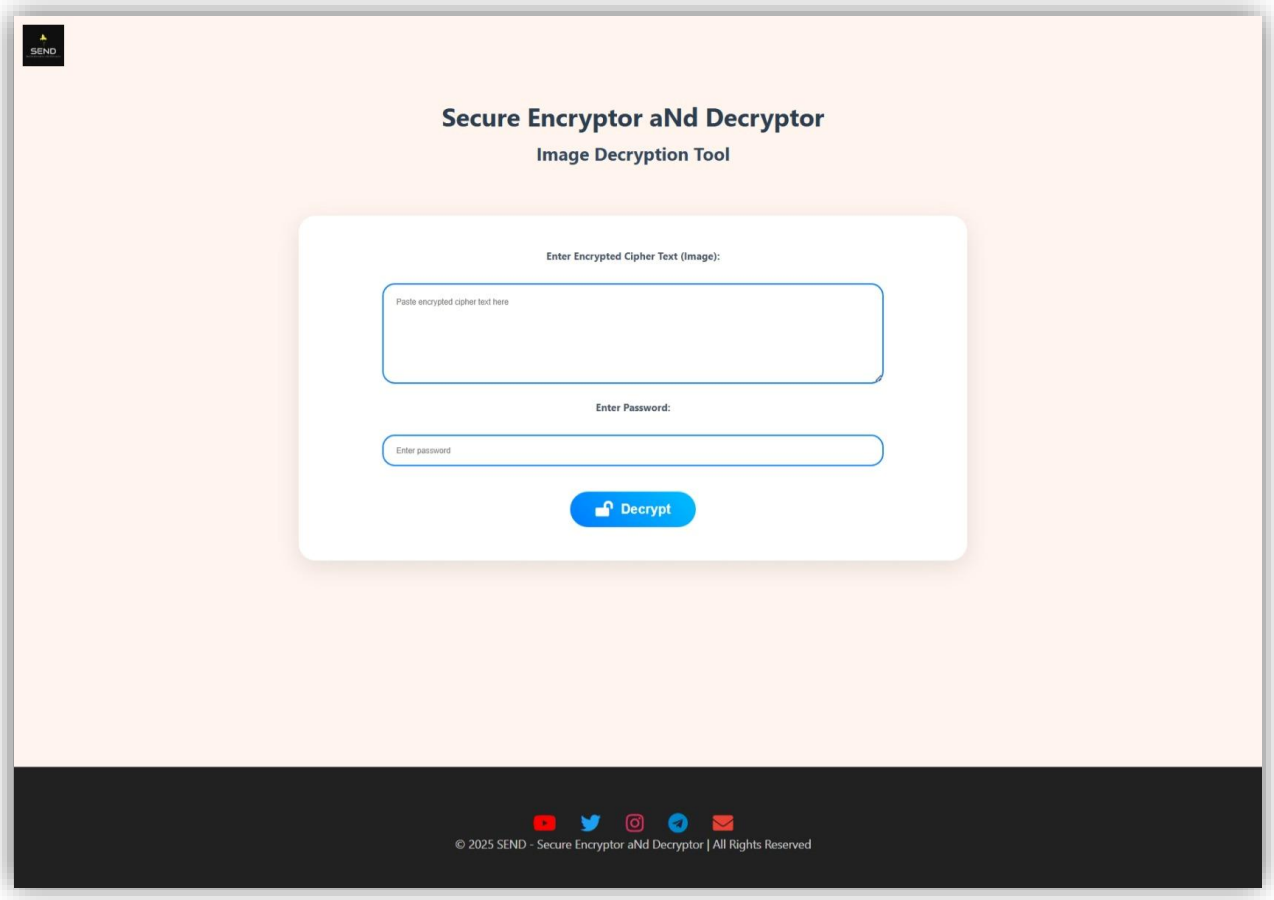
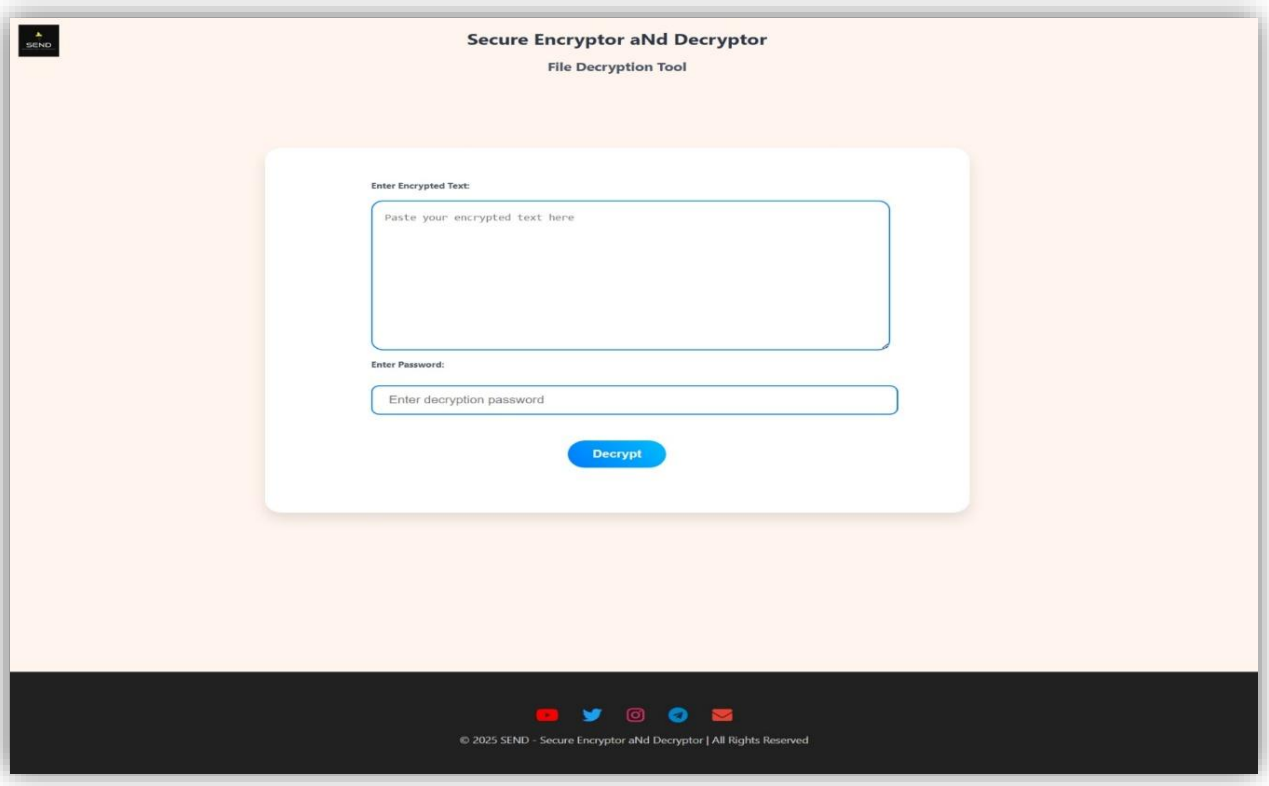


IMAGE DECRYPTION PAGE:



FILE DECRYPTION PAGE:



CONTACT PAGE:

LEARN PAGE:

SEND

Home Contact Us FAQ

Secure Encryptor aNd Decryptor

Learn About Encryption and Decryption

1. What is Data Encryption?

Data encryption is a vital method of protecting sensitive information by converting it from a readable format (plaintext) into an unreadable format (ciphertext). Only individuals with the appropriate decryption key or password can convert the ciphertext back into plaintext. This ensures that unauthorized users cannot make sense of the data even if they gain access to it.

Some key highlights:

- **Primary Types of Encryption:**
  - **Asymmetric Encryption (Public Key Encryption):** Uses two keys—a public key for encryption and a private key for decryption, ensuring secure communication.
  - **Symmetric Encryption:** Utilizes the same key for both encryption and decryption, making it faster but reliant on secure key sharing.
- **Hashing:** Creates a fixed-length hash value for verifying data integrity rather than encryption.

  - **Symmetric Encryption:** Utilizes the same key for both encryption and decryption, making it faster but reliant on secure key sharing.
- **Hashing:** Creates a fixed-length hash value for verifying data integrity rather than encryption.

Why It's Important:
Encryption provides:

- **Authentication:** Verifies the source of the data.
- **Integrity:** Ensures the data hasn't been tampered with during transmission.
- **Non-repudiation:** Prevents the sender from denying their involvement in sending the data.

It's worth noting that while encryption secures the data's contents, it doesn't prevent unauthorized access to systems. Its role is to render stolen data useless to malicious actors, like cybercriminals seeking to exploit sensitive information.

---

2. What is Data Decryption?

Data decryption is the process of converting encrypted data (ciphertext) back into its original readable form (plaintext) using a decryption key or password. This is the counterpart to data encryption, and it's essential for authorized users to access and make sense of the protected information.

How It Works:

- When data is encrypted, an algorithm transforms it into an unintelligible format.
- To reverse this process, a decryption key (unique to the encryption method used) is applied, translating the ciphertext back into plaintext.
- The security of decryption depends on safeguarding this decryption key, as unauthorized access to the key compromises the data.

the ciphertext back into plaintext.

- The security of decryption depends on safeguarding this decryption key, as unauthorized access to the key compromises the data.

Types of Decryption:

1. **Symmetric Decryption:**
   - Uses the same key for both encryption and decryption.
   - Since only one key is involved, it is faster but requires secure key sharing between parties.
   - Example: Decrypting data encrypted with the Advanced Encryption Standard (AES).
2. **Asymmetric Decryption:**
   - Utilizes two keys—a public key for encryption and a private key for decryption.
   - The private key must remain confidential, while the public key can be shared freely.
   - Example: Decrypting a file sent using the RSA algorithm.

Why It's Important:

- **Access Control:** Ensures only authorized individuals can read and use sensitive data.
- **Data Confidentiality:** Prevents unauthorized users from interpreting intercepted or stolen data.

---

3. Types of Algorithms

Both symmetric and asymmetric algorithms are essential in the world of encryption and decryption, but they differ significantly in their approach and use cases.

Symmetric Algorithms

- **Definition:** These algorithms use the same key for both encryption (to convert plaintext to ciphertext) and decryption (to convert ciphertext back to plaintext).
- **Examples:** AES (Advanced Encryption Standard), DES (Data Encryption Standard), and 3DES (Triple Data

- **Definition:** These algorithms use the same key for both encryption (to convert plaintext to ciphertext) and decryption (to convert ciphertext back to plaintext).
- **Examples:** AES (Advanced Encryption Standard), DES (Data Encryption Standard), and 3DES (Triple Data Encryption Standard).
- **Characteristics:**
  - **Speed:** Symmetric algorithms are typically faster than asymmetric algorithms because they involve simpler computations.
  - **Key Management:** The same key must be securely shared between the sender and receiver, which poses a risk if intercepted.
- **Use Cases:** Ideal for encrypting large amounts of data, such as in file storage, database security, or VPNs.

Asymmetric Algorithms

- **Definition:** These algorithms use a pair of keys: a public key (for encryption) and a private key (for decryption). The public key is openly shared, while the private key remains confidential.
- **Examples:** RSA (Rivest-Shamir-Adleman), ECC (Elliptic Curve Cryptography), and Diffie-Hellman.
- **Characteristics:**
  - **Security:** The public-private key mechanism ensures better security for communication between unknown parties.
  - **Complexity:** Asymmetric algorithms are computationally more intensive, making them slower than symmetric algorithms.
- **Use Cases:** Commonly used for secure key exchange, digital signatures, and establishing encrypted communications (e.g., SSL/TLS in websites).

Comparison

Aspect	Symmetric Algorithms	Asymmetric Algorithms

Comparison

Aspect	Symmetric Algorithms	Asymmetric Algorithms
Key Usage	Same key for encryption and decryption	Public key for encryption, private key for decryption
Speed	Faster	Slower due to complex computations
Security	Relies on secure key sharing	More secure for communication between parties
Use Case	Data storage, file encryption	Digital signatures, secure key exchange

Together, symmetric and asymmetric algorithms are often combined in hybrid cryptosystems, where asymmetric encryption is used to share a symmetric key securely, and the symmetric key is then used to encrypt the actual data.

4. DES Algorithm

Data Encryption Standard (DES) is a symmetric-key block cipher developed by IBM in the early 1970s, derived from the Lucifer cipher by Horst Feistel. It was adopted by the National Bureau of Standards (NBS) in 1977 as a Federal Information Processing Standard (FIPS). It became a widely used encryption standard for securing unclassified but sensitive data.

Algorithm Overview

- **Key Size:** 56 bits (effective); originally 64 bits with 8 bits used for parity.
- **Block Size:** 64 bits.
- **Structure:** Balanced Feistel network with 16 rounds.

Process: DES encrypts data by dividing the block into two halves and processing them alternately through

- **Block Size:** 64 bits.
- **Structure:** Balanced Feistel network with 16 rounds.

Process: DES encrypts data by dividing the block into two halves and processing them alternately through substitution and permutation using S-boxes and P-boxes, along with a series of expansion, key mixing, and permutation functions. Decryption reverses the order of the round keys but uses the same algorithm.

NSA Involvement
IBM collaborated with the NSA, who modified the algorithm to strengthen it against differential cryptanalysis while reducing the key size to make it vulnerable to brute-force attacks. Despite concerns over potential backdoors, it was confirmed that the NSA did not insert any deliberate weaknesses into DES.

Security Concerns
DES faced criticism due to its short key length, making it susceptible to brute-force attacks. Key breaking demonstrations include:

- 1997: DESCHALL Project cracked DES using distributed computing.
- 1998: Electronic Frontier Foundation (EFF) built "Deep Crack," a machine that broke DES in 56 hours.
- 2008: Machines like COPACOBANA and RIVYERA significantly reduced the time and cost to break DES.

Cryptanalysis

- Differential Cryptanalysis (Biham and Shamir, 1980s): Requires 2^47 chosen plaintexts.
- Linear Cryptanalysis (Matsui, 1993): Requires 2^43 known plaintexts.
- Davies' Attack: Specialized for DES, requiring 2^50 known plaintexts.

Despite these, brute-force attacks remain the most practical means of breaking DES.

Replacement and Legacy
DES was officially withdrawn in 2005 and replaced by Advanced Encryption Standard (AES) in 2001 after an

Despite these, brute-force attacks remain the most practical means of breaking DES.

Replacement and Legacy
DES was officially withdrawn in 2005 and replaced by Advanced Encryption Standard (AES) in 2001 after an international competition. DES's successor, Triple DES (3DES), improved security by applying DES encryption three times but was slower and is now being phased out.

Impact
DES played a crucial role in developing modern cryptography. It spurred academic research and inspired new cryptanalysis techniques. According to Bruce Schneier, DES "galvanized the field of cryptanalysis" and set the standard for evaluating other block ciphers.

---

5. 3DES Algorithm

Triple DES (3DES) Overview
Triple Data Encryption Algorithm (TDEA), commonly known as Triple DES (3DES), is a symmetric-key block cipher that applies the DES algorithm three times to each data block. Introduced in 1981, it was developed as a more secure alternative to DES due to DES's vulnerable 56-bit key size.

Algorithm Details
3DES operates on 64-bit blocks using key sizes of 112 or 168 bits. It follows a Feistel network structure, with 48 DES-equivalent rounds. The encryption process typically involves three steps: Encrypt with Key 1 (K1), Decrypt with Key 2 (K2), and Encrypt with Key 3 (K3). Decryption reverses the process. This approach enhances security and offers backward compatibility with DES.

Keying Options
There are three keying options:

1. **Option 1 (3TDEA):** Three independent keys (168 bits total). This is the strongest variant.

Keying Options
There are three keying options:

1. **Option 1 (3TDEA):** Three independent keys (168 bits total). This is the strongest variant.
2. **Option 2 (2TDEA):** Two independent keys where K1 = K3 (112 bits effective security). This was disallowed by NIST in 2015.
3. **Option 3:** All three keys are identical (K1 = K2 = K3), equivalent to single DES encryption, and not recommended.

Security
Despite its improvements over DES, 3DES is still vulnerable:

- Meet-in-the-middle attacks reduce its effective security.
- CVE-2016-2183 (Sweet32 attack) exposed vulnerabilities in 3DES due to its small block size (64 bits), making it prone to block collision attacks in protocols like TLS and OpenVPN.

NIST estimates the effective security of 3DES at 112 bits for option 1 and 80 bits for option 2. As a result, NIST deprecated 3DES in 2019, allowing limited use until 2023, and plans full disallowance by 2024.

Use and Deprecation
Although many systems (like EMV payment cards, Microsoft Office, and Mozilla products) historically used 3DES, most have transitioned away from it. OpenSSL removed 3DES by default in 2016, labeling it a weak cipher. It has been largely replaced by the Advanced Encryption Standard (AES), which offers better security and efficiency.

Standards and Compliance
3DES has been defined in several standards, including:

- RFC 1851 (1995)
- ANSI X9.52-1998 (withdrawn in 2008)
- NIST SP 800-67 Rev 2 (2017), withdrawn in 2024

- RFC 1851 (1995)
- ANSI X9.52-1998 (withdrawn in 2008)
- NIST SP 800-67 Rev 2 (2017), withdrawn in 2024

Conclusion
Triple DES provided a practical stopgap solution when DES was no longer secure, but it has now reached the end of its lifecycle due to inherent weaknesses and advancements in cryptanalysis and computational power. Modern cryptographic applications are advised to use stronger algorithms like AES.

---

6. AES Algorithm

Advanced Encryption Standard (AES), also known as Rijndael, is a symmetric-key block cipher selected by the U.S. National Institute of Standards and Technology (NIST) in 2001 to replace the aging Data Encryption Standard (DES). AES was developed by Belgian cryptographers Joan Daemen and Vincent Rijmen, and it became effective as the U.S. Federal Information Processing Standard (FIPS) 197.

Design and Structure

- **Block Size:** 128 bits (fixed)
- **Key Sizes:** 128, 192, or 256 bits
- **Rounds:** 10 (128-bit key), 12 (192-bit key), 14 (256-bit key)

AES uses a Substitution-Permutation Network (SPN), unlike DES, which uses a Feistel network.

Encryption Process
The input (plaintext) is divided into a 4×4 byte matrix, called the state. AES encryption transforms this state over several rounds consisting of:

- **SubBytes:** Non-linear byte substitution using an S-box.

The input (plaintext) is divided into a 4×4 byte matrix, called the state. AES encryption transforms this state over several rounds consisting of:

- **SubBytes:** Non-linear byte substitution using an S-box.
- **ShiftRows:** Row-wise shifting for diffusion.
- **MixColumns:** Column-wise mixing for further diffusion.
- **AddRoundKey:** XOR of the state with the round key.

The final round omits the MixColumns step.

Security
AES has resisted most cryptanalytic attacks:

- Biclique attacks marginally reduce AES-128 complexity from 2^{128} to $2^{126}.1$, but these are still impractical.
- Related-key attacks exist against AES-192 and AES-256 but require unrealistic conditions.
- Side-channel attacks (e.g., cache-timing, differential fault analysis) target AES implementations, not the algorithm itself.

Government and Industry Use
AES is widely used in government, finance, and technology sectors. It's approved by the NSA for securing Top Secret information (with 192 or 256-bit keys). It's standardized in FIPS 197 and ISO/IEC 18033-3.

Performance
AES performs efficiently in both software and hardware. On modern CPUs with AES-NI instructions, it achieves high throughput and low latency, making it ideal for everything from smart cards to cloud servers.

Quantum Resistance

- AES-256 offers reasonable protection against quantum attacks, maintaining strength comparable to AES-128 against classical attacks.

- Side-channel attacks (e.g., cache-timing, differential fault analysis) target AES implementations, not the algorithm itself.

Government and Industry Use
AES is widely used in government, finance, and technology sectors. It's approved by the NSA for securing Top Secret information (with 192 or 256-bit keys). It's standardized in FIPS 197 and ISO/IEC 18033-3.

Performance
AES performs efficiently in both software and hardware. On modern CPUs with AES-NI instructions, it achieves high throughput and low latency, making it ideal for everything from smart cards to cloud servers.

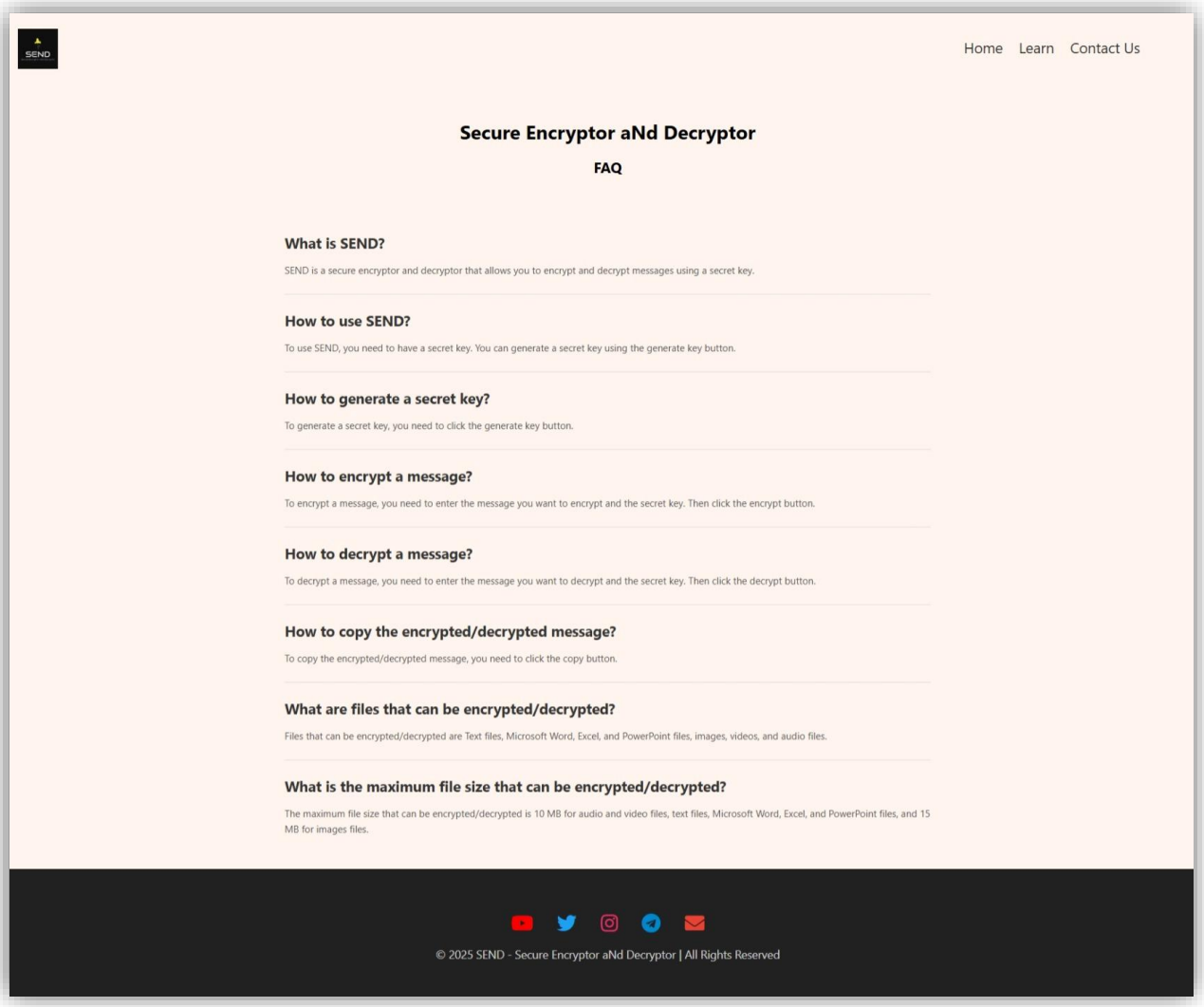
Quantum Resistance

- AES-256 offers reasonable protection against quantum attacks, maintaining strength comparable to AES-128 against classical attacks.
- AES-128 and AES-192 are considered less quantum-resistant due to smaller key sizes.

Validation and Compliance
NIST's Cryptographic Module Validation Program (CMVP) and Cryptographic Algorithm Validation Program (CAVP) ensure AES implementation compliance. FIPS 140-2 validation is required for U.S. government systems handling sensitive data.

© 2025 SEND - Secure Encryptor aNd Decryptor | All Rights Reserved

FAQ PAGE:


SEND
Home
Learn
Contact Us
Secure Encryptor aNd Decryptor
FAQ
What is SEND?
SEND is a secure encryptor and decryptor that allows you to encrypt and decrypt messages using a secret key.
How to use SEND?
To use SEND, you need to have a secret key. You can generate a secret key using the generate key button.
How to generate a secret key?
To generate a secret key, you need to click the generate key button.
How to encrypt a message?
To encrypt a message, you need to enter the message you want to encrypt and the secret key. Then click the encrypt button.
How to decrypt a message?
To decrypt a message, you need to enter the message you want to decrypt and the secret key. Then click the decrypt button.
How to copy the encrypted/decrypted message?
To copy the encrypted/decrypted message, you need to click the copy button.
What are files that can be encrypted/decrypted?
Files that can be encrypted/decrypted are Text files, Microsoft Word, Excel, and PowerPoint files, images, videos, and audio files.
What is the maximum file size that can be encrypted/decrypted?
The maximum file size that can be encrypted/decrypted is 10 MB for audio and video files, text files, Microsoft Word, Excel, and PowerPoint files, and 15 MB for images files.
© 2025 SEND - Secure Encryptor aNd Decryptor | All Rights Reserved

APPENDIX B: CODE

1. index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>SEND - Secure Encryptor aNd Decryptor</title>
    <link rel="icon" href="https://i.ibb.co/0jg38gtW/send.png" type="image/png" />
    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css"
rel="stylesheet"/>
    <style>
        :root {
            --primary-color: #007bff;
            --secondary-color: #faebd4;
            --dark-color: #1a1a1a;
            --text-color: #333;
            --light-text: #fff;
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            display: flex;
            flex-direction: column;
            min-height: 100vh;
```

```
    background-color: #faebd4;
    line-height: 1.6;
}

.container {
    display: flex;
    width: 100%;
    flex: 1;
}

.left {
    background-color: var(--dark-color);
    color: var(--light-text);
    width: 50%;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    position: relative;
    overflow: hidden;
}

.left img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    transition: transform 0.5s ease;
}

.left img:hover {
    transform: scale(1.1);
```

```
}

.right {
    background-color: #faebd4;
    color: var(--text-color);
    width: 50%;
    display: flex;
    flex-direction: column;
    justify-content: center;
    padding: 40px;
}

.right nav {
    display: flex;
    justify-content: flex-end;
    margin-bottom: 40px;
}

.right nav a {
    color: var(--text-color);
    text-decoration: none;
    margin-left: 20px;
    font-size: 18px;
    transition: all 0.3s ease;
    position: relative;
}

.right nav a:after {
    content: '';
    position: absolute;
    width: 0;
```

```
    height: 2px;
    bottom: -5px;
    left: 0;
    background-color: var(--primary-color);
    transition: width 0.3s ease;
}
.right p {
    font-size: 16px;
    line-height: 1.5;
    margin: 0 0 20px 0;
    text-align: justify;
}
.right nav a:hover {
    color: var(--primary-color);
}

.right nav a:hover:after {
    width: 100%;
}

.content-box {
    background-color: #fff5ee;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.content-box:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.15);
```

```
}

h1 {
    font-size: 2.5rem;
    margin-bottom: 1rem;
    color: var(--dark-color);
    text-align: center;
}

h2 {
    font-size: 1.8rem;
    margin-bottom: 1.5rem;
    color: var(--dark-color);
    text-align: center;
}

.contact-info {
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
    align-items: center;
    margin-top: 2rem;
}

.contact-item {
    display: flex;
    align-items: center;
    gap: 1rem;
    font-size: 1.1rem;
    padding: 10px;
    width: 100%;
```

```
    max-width: 400px;
    border-radius: 8px;
    transition: background-color 0.3s ease;
}

.contact-item:hover {
    background-color: rgba(0, 123, 255, 0.1);
}

.contact-item i {
    font-size: 1.5rem;
    width: 40px;
    height: 40px;
    display: flex;
    justify-content: center;
    align-items: center;
    border-radius: 50%;
    background-color: rgba(0, 0, 0, 0.05);
}

.contact-item a {
    color: var(--primary-color);
    text-decoration: none;
    transition: color 0.3s ease;
}

.contact-item a:hover {
    color: #0056b3;
    text-decoration: underline;
}
```

```
.footer {
    position: relative;
    background: #222;
    color: var(--light-text);
    text-align: center;
    padding: 30px 0;
    overflow: hidden;
}

.wave {
    position: absolute;
    top: -20px;
    left: 0;
    width: 100%;
    height: 40px;
    background: url('https://i.ibb.co/w7VLzYM/wave.png');
    background-size: 1000px 40px;
    animation: wave-animation 4s linear infinite;
}

@keyframes wave-animation {
    0% { background-position-x: 0px; }
    100% { background-position-x: 1000px; }
}

.social-icons {
    margin: 20px 0;
}

.social-icons a {
    color: var(--light-text);
```

```
    margin: 0 15px;
    font-size: 24px;
    transition: all 0.3s ease;
    display: inline-block;
}

.social-icons a:hover {
    transform: translateY(-5px);
    color: var(--primary-color);
}

.footer p {
    margin-top: 15px;
    font-size: 14px;
    opacity: 0.8;
}

@media (max-width: 992px) {
    .container {
        flex-direction: column;
    }

    .left, .right {
        width: 100%;
    }

    .left {
        min-height: 300px;
    }

    .right nav {
```

```
        justify-content: center;
        margin-bottom: 20px;
    }

    .right nav a {
        margin: 0 10px;
    }

    h1 {
        font-size: 2rem;
    }

    h2 {
        font-size: 1.5rem;
    }
}

@media (max-width: 576px) {
    .right {
        padding: 20px;
    }

    .contact-item {
        flex-direction: column;
        text-align: center;
    }

    .social-icons a {
        margin: 0 10px;
        font-size: 22px;
    }
```

```
        }
    </style>
    <script src="https://kit.fontawesome.com/a076d05399.js" crossorigin="anonymous"></script>
</head>
<body>
    <div class="container">
        <div class="left">
            <img src="https://i.ibb.co/0jg38gtW/send.png" alt="SEND Logo">
        </div>
        <div class="right content-box">
            <nav>
                <a href="home.html">Home</a>
                <a href="learn.html">Learn</a>
                <a href="contact.html">Contact Us</a>
                <a href="FAQ.html">FAQ</a>
            </nav>
            <h2>Secure Encryptor aNd Decryptor</h2>
            <p>In today's digital era, secure data transmission is essential due to the increasing use of web-based systems for communication and transactions. Our Secure Encryptor aNd Decryptor (SEND) website developed a web application that encrypts and decrypts text, images, audios, videos and various document formats like Text File, PDF, Microsoft Word, Microsoft Excel, Microsoft PowerPoint.
                <br /><br />This website, designed with a client-server architecture, features a user-friendly interface. This website supports DES, Triple DES, AES encryption algorithms, ensuring secure data transmission while maintaining simplicity. A standout feature is its ability to handle multiple file formats, offering versatility for safeguarding diverse types of data during transmission or storage.
                <br /><br /> In conclusion, the Secure Encryptor aNd Decryptor Web Application addresses the need for secure data handling with an efficient, user-friendly solution, emphasizing the importance of integrating robust encryption methods in modern web systems.</p>
        </div>
```

```
    </div>

    <footer class="footer">

        <div class="social-icons">
            <a href="https://www.youtube.com/@send_tool" target="_blank"><i class="fab fa-youtube" style="color: #FF0000"></i></a>
            <a href="https://twitter.com/send_tool" target="_blank"><i class="fab fa-twitter" style="color: #1DA1F2"></i></a>
            <a href="https://www.instagram.com/send_tool" target="_blank"><i class="fab fa-instagram" style="color: #E1306C"></i></a>
            <a href="https://t.me/+Vq1zoqKFqPxkYzk1" target="_blank"><i class="fab fa-telegram" style="color: #0088CC"></i></a>
            <a href="mailto:sendtool315@gmail.com" target="_blank"><i class="fas fa-envelope text-red-500 text-2xl" style="color: #Ea4335"></i></a>
        </div>
        <p>&copy; 2025 SEND - Secure Encryptor aNd Decryptor | All Rights Reserved</p>
    </footer>
</body>
</html>
```